

Aula - \$cond no Aggregate

O IF do MongoDB

Base: restaurants.json | Transformar score em classificação: Bom / Regular / Ruim

1. Contexto - Por que precisamos do \$cond?

- No find() você filtra, no aggregate você CLASSIFICA
- Problema: score 0-10 = Excelente, 11-20 = Bom, >20 = Ruim
- Precisamos criar um campo novo: status = 'Bom' ou 'Ruim'
- No SQL: CASE WHEN score<10 THEN 'Bom' ELSE 'Ruim' END
- No MongoDB: \$cond = IF
- Essencial para relatórios com \$group depois

Restaurante tem:

```
grades: [ { score:2 }, { score:14 }, { score:28 } ]
```

Queremos:

```
{ score:2, status:'Excelente' }  
{ score:14, status:'Bom' }  
{ score:28, status:'Ruim' }
```

Depois agrupar: quantos Excelente, Bom, Ruim por borough?

2. Sintaxe do \$cond - 2 formas

- Forma objeto (recomendada): mais legível
- Forma array: [condição, then, else]
- if: condição com \$eq, \$gt, \$lt, \$gte
- then: valor se verdadeiro
- else: valor se falso
- Pode aninhar \$cond dentro de else para ELSE IF

```
// Forma objeto (didática)
{
  $cond: {
    if: { $lt: ['$grades.score', 10] },
    then: 'Excelente',
    else: 'Ruim'
  }
}
```

```
// Forma array (curta)
{
  $cond: [ { $lt:['$grades.score',10] },
  'Excelente', 'Ruim' ]
}
```

```
// No $project:
{ $project:{ status:{ $cond:{ if:{
  $lt:['$score',10] }, then:'Bom',
  else:'Ruim' } } } }
```

3. Exemplo 1 - Simples: Bom ou Ruim

- \$unwind primeiro para ter score simples
- \$project cria campo status com \$cond
- Score <10 = Excelente, senão Ruim
- Depois pode agrupar por status

```
db.restaurants.aggregate([
  { $unwind: '$grades' },
  { $project: {
    borough: 1,
    score: '$grades.score',
    status: {
      $cond: {
        if: {
          $lt: ['$grades.score', 10] },
          then: 'Excelente',
          else: 'Ruim'
        }
      }
    }
  }},
  { $limit: 5 }
])
// Saída:
// { borough: 'Bronx', score: 2,
//   status: 'Excelente' }
// { borough: 'Bronx', score: 14,
//   status: 'Ruim' }
```

5. \$cond + \$group - Contagem condicional

- Padrão prova: contar quantos A, B, etc por restaurante
- Dentro do \$group, \$sum com \$cond
- Se grade==A soma 1, senão 0
- Cria total, totalA, totalB no mesmo group

```
db.restaurants.aggregate([
  { $unwind: '$grades' },
  { $group: {
    _id: '$restaurant_id',
    nome: { $first: '$name' },
    total: { $sum: 1 },
    totalA: { $sum: { $cond: [{ $eq: ['$grades.grade', 'A']
}, 1, 0] } },
    totalB: { $sum: { $cond: [{ $eq: ['$grades.grade', 'B']
}, 1, 0] } },
    totalRuim: { $sum: { $cond: [{ $gt: ['$grades.score', 30]
}, 1, 0] } }
  } },
  { $sort: { totalB: -1 } },
  { $limit: 5 }
])
// Top 5 com mais notas B
```

6. Pipeline Completo - Relatório por Borough com \$cond

- Objetivo: para cada borough, quantos Excelente/Bom/Ruim?
- Passos: unwind → project com \$cond aninhado → group por borough e classificacao → group de novo para pivot
- Mostra poder de \$cond + \$group

```
db.restaurants.aggregate([
  { $unwind:'$grades' },
  { $project:{
    borough:1,
    status:{
      $cond:{ if:{ $lt:['$grades.score',10] },
    then:'Excelente',
      else:{ $cond:{ if:{ $lt:['$grades.score',20] },
    then:'Bom', else:'Ruim' } } }
    }
  }},
  { $group:{ _id:{ borough:'$borough', status:'$status'
}, total:{ $sum:1 } } },
  { $group:{ _id:'$_id.borough', statuses:{ $push:{
k:'$_id.status', v:'$total' } } } },
  { $project:{ borough:'$_id', resumo:{
$arrayToObject:'$statuses' } } },
  { $sort:{ borough:1 } }
])
// Bronx: { Excelente:500, Bom:300, Ruim:100 }
```

7. Alternativa: \$switch - Mais limpo que \$cond aninhado

- \$switch = vários IF ELSE IF, mais legível que \$cond aninhado
- branches: array de case/then
- default: valor padrão
- Recomendado quando tem mais de 3 condições

```
// Mesmo exemplo com $switch (mais limpo)
{
  $switch:{
    branches:[
      { case:{ $lt:['$grades.score',5] },
        then:'Excelente' },
      { case:{ $lt:['$grades.score',15] }, then:'Bom' },
      { case:{ $lt:['$grades.score',30] }, then:'Regular' }
    ],
    default:'Ruim'
  }
}

// No project:
{ $project:{
  classificacao:{
    $switch:{ branches:[...], default:'Ruim' }
  }
}}
```

4. \$cond Aninhado - IF, ELSE IF, ELSE

- Para 3 ou mais classificações, aninha \$cond no else
- Score <5 = Excelente, <15 = Bom, <30 = Regular, senão Ruim
- Cada else é outro \$cond
- Parece complicado mas é só IF dentro de ELSE

```
db.restaurants.aggregate([
  { $unwind: '$grades' },
  { $project: {
    score: '$grades.score',
    classificacao: {
      $cond: { if: { $lt: ['$grades.score', 5]
    }, then: 'Excelente',
      else: {
        $cond: { if: {
$lt: ['$grades.score', 15] }, then: 'Bom',
          else: {
            $cond: { if: {
$lt: ['$grades.score', 30] }, then: 'Regular',
            else: 'Ruim' }
          }
        }
      }
    }
  }
}, { $limit: 5 }
])
// 2→Excelente, 14→Bom, 28→Regular, 38→Ruim
```

8. Exercícios LAB

- 1-2: Criar campo bom/ruim com \$cond simples
- 3-4: \$cond aninhado 3 níveis Excelente/Bom/Ruim
- 5-6: \$cond dentro de \$group para contar A e B
- 7: Pipeline completo borough + status com \$cond
- 8-9: Desafio só nota A usando \$cond + \$expr
- 10: Relatório final com \$switch + \$group

```
// Roteiro:  
// 0-20min Exemplos 1 e 2 ao vivo  
// 20-40min Exemplo 3 e 4 $cond aninhado  
// 40-60min Exemplo 5 e 6 $cond + $group  
// 60-90min Lab exercícios 1 a 7  
// Casa: 8,9,10 para entregar  
  
// Dica: sempre $unwind antes de usar grades.score no  
$cond
```